CS 498 — Data Management in the Cloud

Assignment 3: Distributed NoSQL, Tunable Consistency & Python API

Design Document

1. Bigtable Design

1.1 Row Key and Sharding Strategy

The BigTable row key format is VIN. With hash-based distribution of rows, records can be distributed evenly among tablets instead of concentrating writes in one lexicographic range. The VIN still stays in the row key after the hash function separator.

Distribution through hashing is the main way of record distribution due to the presence of insert operations in big batches.

1.2 Column Families

Column Family	Example Columns / Purpose
vehicle	make, model, model_year, electric_vehicle_type
location	county, city, state, postal_code, vehicle_location, geoid_2020
technical	electric_range, electric_utility
administrative	cafv_eligibility, legislative_district, dol_vehicle_id

1.3 Hash Sharding vs. Range Sharding

Approach	Advantage	Trade-off
Hash sharding	Evenly distributes rows and writes; reduces hotspot risk.	Less natural for ordered/range queries.
Range sharding	Efficient for ordered/range queries because nearby keys remain together.	Sequential or skewed keys can create hotspots.

2. MongoDB Atlas Architecture

MongoDB Atlas is configured as a three-node replica set. The database is ev_db and the collection is vehicles. The final dataset contains 300,142 vehicle records after loading and removal of test and benchmark documents. Backups are inactive for the assignment.

Network access allows the fixed VM address 34.60.92.184 and the course autograder address 136.64.84.147.

3. Python API and Tunable Consistency

3.1 Write Concern

Endpoint	Write Concern	Purpose
/insert-fast	w=1	Acknowledged primary write optimized for lower latency.
/insert-safe	w='majority'	Acknowledged after replication to a majority of replica-set members.

3.2 Read Preference

Endpoint	Read Preference	Purpose
/count-tesla-primary	PRIMARY	Reads Tesla records from the primary.
/count-bmw-secondary	SECONDARY_PREFERRED	Uses a secondary when available and falls back to the primary.

3.3 Implementation

The Flask application exposes the required endpoints on port 8000. MongoClient reads MONGO_URI from an environment variable so credentials are not hard-coded.

```
fast_collection = vehicles.with_options(write_concern=WriteConcern(w=1))
safe_collection = vehicles.with_options(write_concern=WriteConcern(w='majority'))
primary_collection = vehicles.with_options(read_preference=ReadPreference.PRIMARY)
secondary_preferred_collection = vehicles.with_options(
    read_preference=ReadPreference.SECONDARY_PREFERRED
)
```

4. CAP / Consistency Scenario

The replica-set setup illustrates the compromise between having consistency guarantees and availability in the event of node failure or network failure. In the fast endpoint setup, w is set to 1, meaning the primary does not have to wait for a majority of replicas before acknowledging the write request. However, this results in increased latency with the possibility that the new document might not be replicated by most nodes.

In the safe endpoint, w='majority' where the write is acknowledged when a majority of the replica-set nodes have acknowledged it. This results in higher latency but ensures strong durability/consistency guarantees.

5. Benchmark Results

A benchmark of 500 sequential public HTTP POST requests was executed for each write endpoint. Requests were sent through the public VM endpoint, so network and API overhead were included.

Metric	/insert-fast	/insert-safe	Difference
Requests	500	500	0
Average latency	114.77 ms	130.22 ms	+15.45 ms
Minimum latency	91.16 ms	94.05 ms	+2.89 ms
Maximum latency	941.58 ms	1165.80 ms	+224.22 ms

The majority-write endpoint averaged 130.22 ms versus 114.77 ms for the fast endpoint, an increase of approximately 13.46%. This is consistent with the additional replication acknowledgement required by majority write concern.

6. Validation

Test	Observed Result	Status
Public health endpoint	HTTP 200; status=ok	PASS
POST /insert-fast	HTTP 201; ID returned	PASS
POST /insert-safe	HTTP 201; ID returned	PASS
GET /count-tesla-primary	122,291	PASS
GET /count-bmw-secondary	12,395	PASS
Final collection count	300,142	PASS
Public VM endpoint	34.60.92.184:8000	PASS

7. Conclusion

The system consists of Bigtable hash sharding using MongoDB Atlas replica set architecture along with a Flask/PyMongo API which provides tunable read/write consistency as an explicit example. The benchmark provides evidence for the trade-off between lower-latency acknowledged writes and majority-acknowledged writes. Public validation of the project was achieved by publishing the VM endpoint containing 300,142 vehicle entries.